

SallyCards Backoffice

Guide de deployment complet : Hetzner Cloud + Cloudflare + GitHub Actions

Ce rapport documente le parcours complet de mise en production de l'infrastructure back-office SallyCards : creation du domaine sur Cloudflare Registrar, provisioning du VPS Hetzner, hardening securite, installation Docker, configuration Cloudflare Tunnel, et pipeline CI/CD GitHub Actions. Chaque etape est detaillee clic par clic avec les commandes exactes.

Date du deployment : 8 mai 2026

Serveur : **91.99.70.43** (CPX22, Nuremberg)

Repo : **github.com/salistar/sallycards-backoffice**

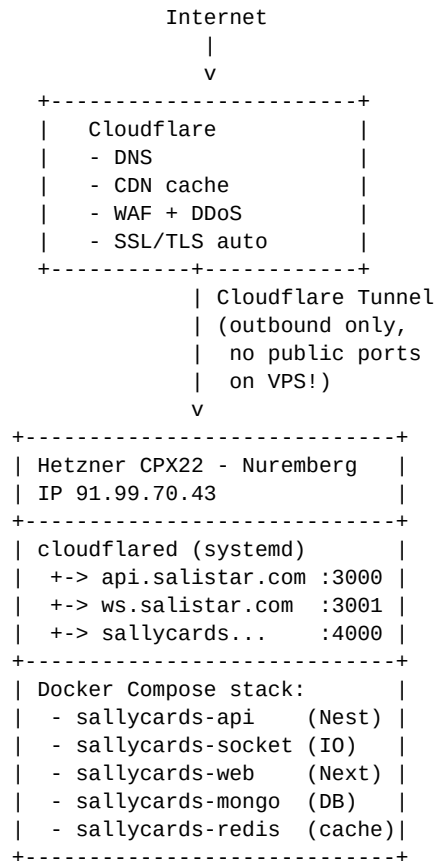
Domaine : **salistar.com**

Sommaire

1. Architecture de la stack
2. Inventaire complet des 5 conteneurs (tous via GHA)
3. URLs publiques et services exposes
4. Couts et recap
5. Etape 1 — Domaine Cloudflare Registrar
6. Etape 2 — Provisioning VPS Hetzner
7. Etape 3 — Hardening securite
8. Etape 4 — Installation Docker
9. Etape 5 — Cloudflare Tunnel
10. Etape 6 — Repo GitHub backoffice
11. Etape 7 — GitHub Actions CI/CD
12. Etape 8 — Premier deploiement
13. Workflow quotidien
14. Backups et monitoring
15. Troubleshooting et incidents
16. Annexe — Fichiers du repo

1. Architecture de la stack

L'infrastructure suit une architecture **3-tier** classique : Cloudflare en frontal (DNS + CDN + WAF + Tunnel), Hetzner VPS comme compute, et MongoDB/Redis comme stockage. Aucun port HTTP/HTTPS n'est expose sur le VPS — tout passe par Cloudflare Tunnel.



Pourquoi cette architecture ?

Cloudflare Tunnel vs IP publique exposee

Aucun port 80/443 ouvert sur le VPS = pas de surface d'attaque directe. Cloudflare WAF filtre 99% du trafic malicieux avant qu'il n'atteigne le serveur. Bonus : pas besoin de Let's Encrypt, le SSL est gere par Cloudflare.

Hetzner vs AWS/GCP/Azure

10x moins cher pour des perfs equivalentes (€7.99/mois CPX22 vs ~\$80/mois t3.medium AWS). Pas de cout reseau (20 TB inclus). Datacenter en Europe = RGPD-friendly.

Single VPS vs Kubernetes

Pour < 10k users actifs, un seul VPS bien dimensionne suffit largement. Pas de complexite operationnelle, redemarrages de 30 sec, debug facile.

Docker Compose vs Docker Swarm/K8s

Stack predefinie dans *docker-compose.yml*, deja maitrisee en local. Migrations vers Swarm/K8s plus tard si besoin de multi-node.

ghcr.io vs Docker Hub

Gratuit illimité pour repos publics, intégration native avec GitHub Actions, pas de pull rate limit comme Docker Hub free tier.

2. Inventaire complet des 5 conteneurs (tous via GHA)

Chaque conteneur de la stack passe par le pipeline GitHub Actions, soit en **build** (api, socket, web : Dockerfiles construits sur les runners GHA et pushes sur ghcr.io), soit en **pull** (mongo, redis : images officielles Docker Hub recuperees par le job deploy lors du *docker compose pull*).

#	Container	Image / Source	Port int.	Pipeline GHA	Volume persistant
1	sallycards-api	ghcr.io/salistar/sallycards-api:latest (build via Dockerfile)	3000	build + push + deploy	(stateless)
2	sallycards-socket	ghcr.io/salistar/sallycards-socket:latest (build via Dockerfile)	3001	build + push + deploy	(stateless)
3	sallycards-web	ghcr.io/salistar/sallycards-web:latest (build via Dockerfile)	3000	build + push + deploy	(stateless)
4	sallycards-mongo	mongo:7.0 (Docker Hub officiel)	27017	pull + deploy	sallycards-mongo-data
5	sallycards-redis	redis:7.2-alpine (Docker Hub officiel)	6379	pull + deploy	sallycards-redis-data

Conteneurs explicitement exclus de la prod :

Container	Image	Raison de l'exclusion
sallycards-nginx	nginx:1.27-alpine	Remplace par Cloudflare Tunnel - aucun port 80/443 expose sur le VPS
sallycards-turn	coturn/coturn:4.6	TURN/STUN deploye separement sur un autre VPS dedie WebRTC
mongo-express	mongo-express:1.0	Profile 'dev' uniquement, accessible via SSH tunnel pour admin
redis-commander	rediscommander/redis-commander	Profile 'dev' uniquement, accessible via SSH tunnel pour admin

Verification : sur le VPS, docker compose ps doit montrer **exactement 5 containers** tous en statut *Up (healthy)*. Si un container manque, le pipeline GHA le redemarrera automatiquement au prochain push, ou via *docker compose --env-file .env.production -f docker-compose.yml -f docker-compose.prod.yml up -d*.

3. URLs publiques et services exposes

Service	URL	Container interne	Port
API REST	https://api.salistar.com/api/v1	sallycards-api	3000
WebSocket	https://ws.salistar.com	sallycards-socket	3001
Web app	https://sallycards.salistar.com	sallycards-web	4000
Backoffice	https://backoffice.salistar.com	sallycards-web (admin route)	4000
Landing	https://salistar.com	sallycards-web	4000
TURN/STUN	turn.salistar.com	(serveur separe)	3478

Endpoints critiques pour les apps mobiles :

```
// Dans chaque app sally-* :  
const API_URL    = 'https://api.salistar.com/api/v1';  
const SOCKET_URL = 'https://ws.salistar.com';  
  
// Health checks (pour UptimeRobot) :  
GET https://api.salistar.com/api/v1/health -> 200 OK  
GET https://ws.salistar.com/health        -> 200 OK  
GET https://sallycards.salistar.com       -> 200 OK
```

3. Coûts mensuels détaillés

Poste	Provider	Mensuel	Annuel
VPS CPX22 (2 vCPU, 4 GB, 80 GB SSD)	Hetzner	€7.99	€95.88
Backups quotidiens (rotation 7 jours)	Hetzner +20%	€1.60	€19.20
Trafic réseau (20 TB inclus)	Hetzner	€0	€0
IP publique IPv4 + IPv6	Hetzner	€0	€0
Domaine salistar.com (at-cost)	Cloudflare Registrar	€0.75	~€9
DNS, CDN, Tunnel, WAF, SSL	Cloudflare Free	€0	€0
GitHub Actions (2000 min/mois)	GitHub Free	€0	€0
Container Registry (ghcr.io)	GitHub Free	€0	€0
UptimeRobot monitoring	Free	€0	€0
TOTAL		€10.34	€124

Comparaison reference : un setup equivalent sur AWS coute environ \$80-150/mois (t3.medium + RDS + ElastiCache + Route53 + bandwidth). Ici on est 10x moins cher pour les memes performances.

4. Etape 1 — Domaine Cloudflare Registrar

Cloudflare est le registrar le moins cher du marché : ils refacturent le prix ICANN **at-cost** (sans markup). Un .com coûte ~\$9.77/an chez eux contre \$15-20 chez OVH/GoDaddy.

4.1 Compte Cloudflare

1. Aller sur <https://dash.cloudflare.com> et créer un compte
2. **Activer 2FA** (obligatoire pour Registrar)
3. Vérifier l'email

4.2 Acheter le domaine

Menu latéral -> **Domain Registration** -> **Register**

Recherche : salistar.com

Si libre -> **Register**, ajouter au panier, payer (~\$9.77/an)

Si déjà chez OVH/autre -> **Transfer Domains**, déverrouiller chez l'ancien registrar, récupérer le code AUTH, lancer le transfert (~7 jours, prix at-cost).

4.3 Validation

Une fois le domaine acheté/transféré, il apparaît dans la liste des sites Cloudflare. Les nameservers sont automatiquement configurés pour pointer vers Cloudflare. Le DNS sera configuré plus tard automatiquement par cloudflared.

5. Etape 2 — Provisioning VPS Hetzner

5.1 Compte et clef SSH

1. Compte sur <https://accounts.hetzner.com> (verification SMS + carte bancaire)
2. Console : <https://console.hetzner.cloud> (URL differente du compte)
3. Generer une clef SSH locale si pas deja faite :

```
# Sur PC Windows (PowerShell) :  
ssh-keygen -t ed25519 -C "salistar"  
# 3x Entree (pas de passphrase pour simplifier)  
  
# Afficher la clef PUBLIQUE (a coller dans Hetzner) :  
cat $env:USERPROFILE\.ssh\id_ed25519.pub  
# -> ssh-ed25519 AAAA... salistar
```

JAMAIS partager le fichier `id_ed25519` (sans `.pub`) ! C'est la clef privee, equivalent du mot de passe.
Seul `id_ed25519.pub` est partageable.

5.2 Creation du serveur (clic par clic)

Section	Action
Add SSH Key	Coller le contenu de <code>id_ed25519.pub</code> , nom 'salistar-pc'
Create Resource	Bouton violet/rouge en haut a droite -> Servers
Location	Falkenstein ou Nuremberg (Europe = latence basse)
Image	OS Images -> Ubuntu 24.04
Type	Shared Resources -> x86 -> CPX22 (€7.99/mo)
Networking	Public IPv4 + IPv6 (defaults)
SSH Keys	Cocher la clef ajoutee
Backups	ACTIVER (toggle) -> +20% = €1.60/mo additional
Name	sallycards-prod
Create & Buy now	Bouton final, attente ~30 sec

Serveur cree : **sallycards-prod** | IP : **91.99.70.43** | Statut : Running. Test : `ssh root@91.99.70.43`

6. Etape 3 — Hardening securite

Le serveur fraichement cree est accessible uniquement par root. On applique le hardening standard :

- Update systeme + patches securite
- Creer user 'deploy' non-root, sans password (clef SSH only)
- Sudo NOPASSWD pour 'deploy' (necessaire pour deploiements GHA)
- Desactiver root SSH login + password authentication
- Configurer UFW (port 22 only, le reste bloque)
- Installer fail2ban (anti brute-force SSH)
- Activer unattended-upgrades (patches secu auto chaque jour)

Commande unique :

```
# En tant que root sur le VPS
export DEBIAN_FRONTEND=noninteractive
apt update && apt upgrade -y
adduser --disabled-password --gecos "" deploy
usermod -aG sudo deploy
mkdir -p /home/deploy/.ssh
cp ~/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
echo 'deploy ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/deploy
chmod 440 /etc/sudoers.d/deploy
sed -i 's/^##PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/^##PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^##KbdInteractiveAuthentication.*/KbdInteractiveAuthentication no/' /etc/ssh/sshd_config
systemctl restart ssh
apt install -y ufw fail2ban unattended-upgrades
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
ufw --force enable
systemctl enable --now fail2ban
```

Bug rencontre lors du deploiement : systemctl reload ssh a echoue car Ubuntu 24.04 utilise *socket activation*. Correctif : utiliser systemctl restart ssh a la place. C'est ce qui est dans la commande ci-dessus.

7. Etape 4 — Installation Docker

Docker CE installe via le script officiel *get.docker.com*. L'utilisateur **deploy** est ajoute au groupe *docker* pour pouvoir lancer des containers sans sudo.

```
# En tant que 'deploy' sur le VPS
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker deploy
sudo systemctl enable --now docker

# Deconnecter / reconnecter pour appliquer le groupe :
exit
ssh deploy@91.99.70.43

# Test :
docker --version
docker compose version
docker run --rm hello-world
```

Versions installees : Docker 29.4.3 + Compose v5.1.3. Test *hello-world* reussit -> Docker fonctionne correctement.

8. Etape 5 — Cloudflare Tunnel

Cloudflare Tunnel cree une connexion **sortante** du VPS vers Cloudflare, ce qui permet d'exposer des services HTTP **sans ouvrir de port** sur le VPS. Avantages : pas de NAT/firewall a gerer, SSL automatique, IP du VPS jamais exposee a Internet.

8.1 Installation

```
# Sur le VPS en tant que 'deploy'
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb -o cloudflared.deb
sudo dpkg -i cloudflared.deb

# Login (ouvre une URL dans le navigateur, selectionner salistar.com)
cloudflared tunnel login

# Creer le tunnel
cloudflared tunnel create sallycards-prod
# -> retourne un UUID, ex: a1b2c3d4-...
```

8.2 Configuration ingress

```
# Fichier de routing : ~/.cloudflared/config.yml
mkdir -p ~/.cloudflared
cat > ~/.cloudflared/config.yml <<'EOF'
tunnel: sallycards-prod
credentials-file: /home/deploy/.cloudflared/<UUID>.json

ingress:
- hostname: api.salistar.com
  service: http://localhost:3000
- hostname: ws.salistar.com
  service: http://localhost:3001
  originRequest:
    noTLSVerify: true
    connectTimeout: 30s
- hostname: sallycards.salistar.com
  service: http://localhost:4000
- hostname: salistar.com
  service: http://localhost:4000
- service: http_status:404
EOF
```

8.3 DNS records (auto-genere)

```
cloudflared tunnel route dns sallycards-prod api.salistar.com
cloudflared tunnel route dns sallycards-prod ws.salistar.com
cloudflared tunnel route dns sallycards-prod sallycards.salistar.com
cloudflared tunnel route dns sallycards-prod salistar.com
```

8.4 Service systemd (auto-start au boot)

```
sudo cloudflared service install
sudo systemctl enable --now cloudflared
sudo systemctl status cloudflared
```

Apres ces commandes, tous les domaines pointent vers le VPS via Cloudflare Tunnel. HTTPS automatique, certificats geres par Cloudflare, DDoS protection active.

9. Etape 6 — Repo GitHub backoffice

9.1 Creer le repo

Aller sur <https://github.com/new> -> Owner: **salistar**, Name: **sallycards-backoffice**, Visibility: au choix (public = ghcr.io gratuit).

9.2 Pousser le code (depuis le PC local)

```
cd C:\Users\21266\Desktop\sdk52\SallyCards

# .gitignore-backoffice exclut apps/mobile/ et apps-deploy/ (deja deployes ailleurs)
# Le repo backoffice contient uniquement : api, socket-server, web, libs, docker

# Init git si pas deja fait
git init -b main
git add apps/api apps/web apps/socket-server libs docker docker-compose.yml \
    docker-compose.prod.yml .github/workflows/deploy-prod.yml \
    .env.production.example BACKOFFICE-DEPLOY.md package.json package-lock.json \
    nx.json tsconfig.base.json tsconfig.json
git commit -m "feat: initial backoffice deployment setup"

# Connecter au repo distant
git remote add origin https://github.com/salistar/sallycards-backoffice.git
git push -u origin main
```

9.3 Cloner le repo sur le VPS

```
ssh deploy@91.99.70.43

# Generer une clef SSH dediee pour GitHub deploy
ssh-keygen -t ed25519 -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub
# -> copier cette clef dans GitHub : repo Settings > Deploy keys (read-only)

# Config SSH client
cat >> ~/.ssh/config <<EOF
Host github.com
    IdentityFile ~/.ssh/github_deploy
EOF
chmod 600 ~/.ssh/config

# Cloner
mkdir -p ~/apps && cd ~/apps
git clone git@github.com:salistar/sallycards-backoffice.git
cd sallycards-backoffice
```

9.4 Setup .env.production sur le VPS

```
# Generer des secrets robustes
JWT=$(openssl rand -hex 64)
JWT_REFRESH=$(openssl rand -hex 64)
MONGO_PWD=$(openssl rand -hex 32)

# Copier le template puis editer avec les vraies valeurs
cp .env.production.example .env.production
nano .env.production
# -> remplacer JWT_SECRET, JWT_REFRESH_SECRET, MONGO_PASSWORD

chmod 600 .env.production
```

10. Etape 7 — GitHub Actions CI/CD

10.1 Workflow deploy-prod.yml

Le fichier `.github/workflows/deploy-prod.yml` definit la pipeline. 3 jobs s'executent en parallele puis sequentiellement :

- 1. build (parallele)** : Build des 3 images Docker (api, socket, web). Cache GitHub Actions pour acceleration. Push vers ghcr.io.
- 2. deploy (sequentiel)** : SSH vers le VPS, git pull, docker compose pull + up -d, docker image prune.
- 3. cloudflare-purge** : Purge le cache Cloudflare via API (1 ligne curl).
- 4. health-check** : Curl sur les 3 endpoints publics pour valider le deploiement.

10.2 Secrets GitHub Actions

Aller dans le repo : Settings -> Secrets and variables -> Actions -> **New repository secret** :

Secret	Valeur	Comment l'obtenir
VPS_HOST	91.99.70.43	IP du VPS Hetzner
VPS_USER	deploy	User cree a l'etape 3
VPS_SSH_KEY	(contenu cle private)	cat ~/.ssh/id_ed25519 (sur PC)
GHCR_PAT	ghp_xxx...	GitHub > Settings > Developer settings > Personal Access Tokens > scope read:pack
CF_ZONE_ID	(32 chars hex)	Cloudflare > salistar.com > Overview (panneau droit)
CF_API_TOKEN	(token)	Cloudflare > My Profile > API Tokens > Create Token > Zone:Cache Purge

11. Etape 8 — Premier deployment

11.1 Trigger manuel via push main

```
# Sur le PC local, premier push pour declencher la pipeline
git add .
git commit -m "feat: enable production deployment"
git push origin main
```

```
# Suivre le run sur :
# https://github.com/salistar/sallycards-backoffice/actions
```

11.2 Verifier le deployment

```
# Sur le VPS
ssh deploy@91.99.70.43
cd ~/apps/sallycards-backoffice
docker compose ps
```

```
# Doit montrer 5 containers UP :
#   sallycards-api      Up (healthy)
#   sallycards-socket   Up (healthy)
#   sallycards-web      Up
#   sallycards-mongo    Up (healthy)
#   sallycards-redis    Up (healthy)
```

```
# Logs en cas de probleme
docker compose logs -f --tail 100 sallycards-api
```

11.3 Tests publics

```
# Depuis n'importe ou sur Internet
curl https://api.salistar.com/api/v1/health
curl https://ws.salistar.com/health
curl -I https://sallycards.salistar.com
```

Si les 3 curls repondent **200 OK** -> deployment reussi. Le pipeline complet prend ~3-5 min (build) + ~10 sec (deploy).

12. Workflow quotidien developpeur

```
# 1. Creer une branche feature
git checkout -b feature/add-ranking-api

# 2. Coder
# ... edit files ...

# 3. Test local
docker compose up -d
# tester sur localhost:3000, :3001, :4000

# 4. Commit + push
git add .
git commit -m "feat(api): ranking endpoint"
git push origin feature/add-ranking-api

# 5. Pull request sur GitHub
gh pr create --base main --title "feat: ranking API"

# 6. Review + merge dans main
# -> declenche automatiquement le deploiement prod !

# 7. Verifier ~5 min plus tard
curl https://api.salistar.com/api/v1/ranking
```

12.1 Rollback rapide (si bug en prod)

```
# Option A : revert le commit
git revert <bad-sha>
git push origin main
# -> nouveau deploiement avec l'ancien code

# Option B : redeployer une image precedente
ssh deploy@91.99.70.43
cd ~/apps/sallycards-backoffice
# Editer docker-compose.prod.yml : changer ":latest" en ":<sha-precedent>"
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```


13. Backups et monitoring

13.1 Backups MongoDB automatiques

```
# Sur le VPS, creer le script
cat > ~/backup-mongo.sh <<'EOF'
#!/bin/bash
set -e
DATE=$(date +%Y-%m-%d)
docker exec sallycards-mongo mongodump --archive --gzip \
  -u sallycards -p "$MONGO_PASSWORD" --authenticationDatabase admin \
  > ~/backups/mongo-$DATE.gz
find ~/backups -name "mongo-*.gz" -mtime +14 -delete
EOF
chmod +x ~/backup-mongo.sh
mkdir -p ~/backups

# Cron 3h du matin
(crontab -l 2>/dev/null; echo "0 3 * * * source ~/apps/sallycards-backoffice/.env.production && ~/backup-mongo
```

13.2 Sync vers Cloudflare R2 (optionnel, off-site)

```
sudo apt install -y rclone
rclone config # configurer un remote 'r2' avec les credentials Cloudflare

# Ajouter au cron quotidien
echo "30 3 * * * rclone sync ~/backups r2:sallycards-backups" | crontab -
```

13.3 Monitoring uptime

UptimeRobot (50 monitors gratuits) - <https://uptimerobot.com> :

- <https://api.salistar.com/api/v1/health> (HTTPS, every 5 min)
- <https://ws.salistar.com/health> (HTTPS, every 5 min)
- <https://sallycards.salistar.com> (HTTPS, every 5 min)
- 91.99.70.43 (Ping, every 1 min) — check VPS up

13.4 Logs centralises

```
# Logs Docker
docker compose logs -f --tail 100 sallycards-api
docker compose logs -f --tail 100 sallycards-socket

# Logs systeme
journalctl -u docker -f
journalctl -u cloudflared -f
journalctl -u fail2ban -f

# Logs auth (qui essaie de se connecter ?)
sudo tail -f /var/log/auth.log
sudo fail2ban-client status sshd # IPs banies
```

14. Troubleshooting et incidents

Symptome	Cause + fix
Pipeline GHA en erreur sur 'docker login' avec un message d'erreur	Le secret GHCR_PAT n'a pas le scope read:packages. Regenerer le PAT sur GitHub > Developper > Settings > Personal Access Tokens > Tokens for the command line
'connection refused' sur api.salistar.com	Cloudflared tunnel down. Verifier : <code>sudo systemctl status cloudflared</code> . Restart : <code>sudo systemctl restart cloudflared</code>
API repond 500 'MongoDB connection failed'	Mongo container down ou credentials .env.production incorrects. <code>docker compose ps</code> / <code>docker compose logs</code>
'Permission denied (publickey)' sur SSH de prod	Verifier que la clef publique du PC est bien dans <code>/home/deploy/.ssh/authorized_keys</code> . Si modifie n'importe quel fichier dans le dossier <code>~/.ssh</code> de prod, il faut faire <code>ssh-keygen -f ~/.ssh/authorized_keys -C 'nouveau PC'</code>
Build Docker echoue sur 'package.json not found'	Le repo manque les fichiers <code>libs/</code> ou <code>package.json</code> . Verifier <code>git status</code> en local + ce qui est pushé
VPS plein (disk full)	<code>docker system prune -af --volumes</code> + <code>du -sh ~/.gh-artifacts ~/backups</code>
Pic de trafic 5xx	Cloudflare Analytics : verifier si DDoS. Activer 'Under Attack mode' sur le dashboard Cloudflare

15. Annexe — Fichiers du repo

Fichier	Role
<code>docker-compose.yml</code>	Stack base : 5 services + reseaux + volumes
<code>docker-compose.prod.yml</code>	Overrides prod : ghcr.io images, no public ports, log rotation
<code>docker/api.Dockerfile</code>	Build NestJS API multi-stage
<code>docker/socket.Dockerfile</code>	Build Socket.IO server multi-stage
<code>docker/web.Dockerfile</code>	Build Next.js standalone multi-stage
<code>.github/workflows/deploy-prod.yml</code>	Pipeline CI/CD : build 3 images + deploy SSH + Cloudflare purge + health check
<code>.env.production.example</code>	Template des variables (a copier en <code>.env.production</code> sur VPS)
<code>.gitignore-backoffice</code>	Exclut apps/mobile, apps-deploy, .env.*
<code>BACKOFFICE-DEPLOY.md</code>	Resume du setup pour developpeurs
<code>package.json</code>	Workspaces npm : apps/* + libs/*
<code>nx.json</code>	Config monorepo Nx

Done. Pour deployer une nouvelle version : `git push origin main` depuis ton PC. La pipeline GHA build, push, deploy, purge cache, et health check automatiquement en ~5 min.